

CalcEngine

An Arbitrary Precision Calculator

By Praneeth Kodavati
CS24BTECH11034

April 2025

IIT Hyderabad

Unleashing Precision: What is CalcEngine?

CalcEngine is a custom-built Java library designed to perform arithmetic operations with arbitrary precision on both integers and floating-point numbers. Unlike standard Java types, which are limited by fixed-size data types, CalcEngine leverages string-based representations and algorithmic techniques to handle numbers of virtually any size or precision. This engine supports addition, subtraction, multiplication, and division for both whole numbers and decimals, ensuring accuracy in calculations that surpasses native language constraints.

Breaking Shackles - Java's Built-in Limits

Java's standard `int` type can only store values up to $2,147,483,647 (2^{31} - 1)$, which is just 10 digits. This makes it impossible to work with extremely large numbers, such as those found in scientific computations. Moreover, Java's built-in types like `int` and `double` handle truncation and rounding in ways that are not always under your control: for example, casting a float to an int simply chops off the decimal part, and integer division automatically discards any remainder. With **CalcEngine**, you can perform calculations on numbers of any size, and you have full control over how and when truncation happens—enabling precision and flexibility that Java's native types just can't offer.

Why the Name “CalcEngine”?

A Name That Speaks for Itself

The term “CalcEngine” is both intuitive and descriptive: “Calc” stands for calculation, while “Engine” denotes the core computational component driving the system. By choosing “CalcEngine” the project immediately communicates its purpose and aligns with established terminology, making it accessible and familiar to readers and users.

Project Contents

The **CalcEngine** is organized into several components under a main directory. The key subdirectory is `arbitraryarithmetic`, which contains the core arithmetic logic. The full list of files and folders is as follows:

- `arbitraryarithmetic/`
 - `aarithmetic.jar`
 - `AFloat.java`
 - `AFloat.class`
 - `AInteger.java`
 - `AInteger.class`
- `build.xml`
- `CalcEngine.py`
- `MyInfArith.java`
- `MyInfArith.class`
- `Report.tex`
- `README.md`

File Descriptions

aarithmic.jar This is a compiled Java archive (JAR) containing the core logic for arbitrary precision arithmetic. It may be used by other applications or scripts to perform operations on large numbers beyond standard data type limits.

AFloat.java and AFloat.class These files implement and compile a custom class to represent and operate on floating-point numbers with arbitrary precision. The `.java` file contains the source code, while the `.class` file is the compiled bytecode.

AInteger.java and AInteger.class Similar to `AFloat`, these files handle operations for large integers. This class supports basic arithmetic, comparison, and normalization of integer values represented as strings.

build.xml This is an Ant build configuration file that automates the compilation and packaging of the project, including generating JAR files and running tests.

CalcEngine.py A Python script which compiles and runs the main Java file, "MyInfArith.java".

MyInfArith.java and MyInfArith.class This is a java file taking 4 command line arguments and does the necessary operation by extracting functions from `AFloat` and `AInteger`.

Report.tex A LaTeX document containing the written report for the project, which could include the design, implementation, and results.

README.md This Markdown file guides the users how to use this package as a library in detail.

Design Choice - Why Static Method ?

When designing `CalcEngine`, the goal was to create an experience similar to using a physical calculator: you provide two numbers and select an operation, and the calculator returns the result. To achieve this, I implemented each arithmetic operation as a **static method**. This design means you do not need to create an object for every calculation-instead, you simply call the relevant method with your input values.

The `AInteger` class acts as a dedicated integer calculator, handling all operations involving whole numbers with arbitrary precision. Similarly, the `AFloat` class serves as a floating-point calculator, managing calculations for numbers with decimal points. Both classes expose static methods for addition, subtraction, multiplication, and division, making them easy to use and consistent in their interface.

This approach keeps the usage intuitive and modular: you extract the operation you need from the appropriate class, supply the numbers, and immediately receive the answer. By separating integer and floating-point logic, the code remains organized and maintainable, while the static method pattern ensures that `CalcEngine` is efficient and straightforward to use-just like a real calculator, but with no limits on number size or precision.

Exploring the Structure of AInteger

The `AInteger` class is the backbone of integer arithmetic in `CalcEngine`, meticulously designed for arbitrary precision calculations. At its core, the class stores the integer value as a string, allowing it to represent numbers far beyond the limits of standard Java types.

The class begins with several constructors: a default constructor initializes the value to zero, another accepts a string for direct initialization, and a copy constructor allows duplication of existing `AInteger` objects. A simple getter method provides access to the internal string representation, while a static `parse` method offers convenient instantiation from user input.

For internal processing, `AInteger` includes a suite of static helper methods. The `cleanLeadingZeroes` method ensures that numbers like “000123” are normalized to “123,” supporting both positive and negative values. The `reverseString` utility is used for digit-wise operations, while `stringCompare` enables magnitude comparison between two numbers. Sign handling is managed by `signDecider`, which determines whether each operand is positive or negative, and `magnitudeMaker`, which extracts absolute values and strips unnecessary zeros.

Arithmetic operations are implemented as protected static methods that operate directly on string representation. Addition, subtraction, multiplication, and division handle positive numbers at the digit level, carefully managing carries, borrows, and sign logic. Public methods for each operation take care of sign management, magnitude extraction, and the invocation of the appropriate core arithmetic routine. The result is always truncated to 30 in default settings but you can change that any time and returned as a new `AInteger` object.

To change the precision you can just change the static attribute named “precision” to any positive integer you want. The main aim of this class is to help in making the large computation easier. The logic behind the methods for implementing arithmetic operations is very simple - *The methods you learnt in your school*

How AFloat is made using AIInteger

AFloat is a floating-point number class, but instead of using normal floating-point math, it uses the `AInteger` class to do all the calculations as if they are big integers. Here is how it works for each operation:

Addition and Subtraction: First, both numbers are split into their whole part and decimal part (like for 1.23, the whole part is 1 and decimal part is 23). If one number has fewer decimal digits, we add zeros at the end to make both decimal parts the same length. This is called padding. For example, to add 1.2 and 3.45, we change 1.2 to 1.20. Then, we join the whole part and decimal part together (1.20 becomes 120, 3.45 becomes 345) and use `AInteger` to add or subtract them. After getting the result, we put the decimal point back in the correct place.

Multiplication: For multiplication, we remove the decimal points from both numbers, so 1.2 becomes 12 and 3.4 becomes 34. We multiply these numbers using `AInteger` ($12 \times 34 = 408$). Then, we count how many decimal digits there were in total (1 in 1.2 and 1 in 3.4, so 2 in total) and put the decimal point back in the answer (408 becomes 4.08).

Division: For division, we first remove the decimals and then add extra zeros to the end of the numerator to keep the answer precise. For example, if we want 30 digits of precision, we add 30 zeros. Then we divide using `AInteger`. After division, we put the decimal point back so that the answer has the right number of decimal digits.

Padding with zeros is important because it makes sure both numbers line up correctly for addition and subtraction. Removing the decimal point lets us use integer math for all calculations, and putting the decimal back at the end gives us the correct floating-point answer. All the tricky decimal stuff is handled by just moving the decimal point and using `AInteger` for the actual math.

Attributes and methods defined in AIInteger

The `AInteger` class is built to handle integers of any size using string manipulation, allowing arithmetic beyond the limits of built-in types. Here we break down its core logic using code snippets and explanations.

Attributes

- `private String number`: Holds the integer value as a string for arbitrary precision. This is made Private to avoid accidental changes to it from outside .

Public Methods

- `AInteger(String s)`: Constructor from string.
- `AInteger(AInteger original)`: Copy constructor.
- `String getter()`: Returns the internal number string.
- `static AIInteger parse(String s)`: Parses a string to an `AInteger`.
- `static AIInteger addition(AInteger, AIInteger)`: Adds two `AInteger` objects.
- `static AIInteger subtraction(AInteger, AIInteger)`: Subtracts one `AInteger` from another.
- `static AIInteger multiplication(AInteger, AIInteger)`: Multiplies two `AInteger` objects.
- `static AIInteger division(AInteger, AIInteger)`: Divides one `AInteger` by another.

Helper Functions (Mostly Private/Protected)

- `validateIntegerInput(String s)`: Checks if input is a valid integer string.
- `cleanLeadingZeroes(String number)`: Removes unnecessary leading zeros.
- `reverseString(String number)`: Reverses a string for easier digit-wise operations.
- `stringCompare(String n1, String n2)`: Compares two numeric strings.
- `signDecider(String n1, String n2)`: Determines the sign (+ or -) of inputs.
- `magnitudeMaker(String n1, String n2, String s1, String s2)`: Returns absolute values.
- `additionOfPositiveIntegers(String, String)`: Adds two positive numbers as strings.
- `subtractionOfPositiveIntegers(String, String)`: Subtracts two positive numbers as strings.
- `multiplicationOfPositiveIntegers(String, String)`: Multiplies two positive numbers as strings.
- `divisionOfPositiveIntegers(String, String)`: Divides two positive numbers as strings.

Why Some Methods Are Private/Protected

Helper and utility methods are private or protected to keep the internal logic encapsulated and prevent misuse from outside the class. Only the main arithmetic operations are public so that users interact with a clean and safe interface.

Implementation of AIInteger

Addition Simulation

1. **Align Lengths**: Pad shorter number with leading zeros

num1: "1234", num2: "56" → pad num2 → "0056"

2. **Digit Processing**: Add right-to-left with carry

4+6=10 → write 0, carry 1
3+5+1=9 → write 9
2+0=2 → write 2
1+0=1 → write 1

3. Result Construction: Reverse and clean zeros

Reversed sum: "0921" → reverse → "1290" → clean → "1290"

Subtraction Simulation

- Compare magnitudes using `stringCompare()` to make sure that difference is positive
- Process digits with borrow handling:

2-5 → borrow 1 → 12-5=7, next digit reduced by 1

- Handle sign based on magnitude comparison

Multiplication Process

```
protected static String multiplicationOfPositiveIntegers(String num1, String num2) {
    String product = "0";
    for(each digit in num2) {
        // Multiply num1 with current digit
        // Shift partial result left with zeros
        // Add to accumulated product
    }
    return product;
}
```

Division Algorithm

Implements long division through repeated subtraction:

```
public static String divisionOfPositiveIntegers(String dividend, String divisor) {
    String tempDividend = "";
    for(each digit in dividend) {
        tempDividend += digit;
        // Find maximum multiplier (factor) where divisor * factor is less than tempDividend
        // Subtract divisor * factor from tempDividend
        // Append factor to quotient
    }
}
```

Sign Handling Tables

Sign1	Sign2	Operation
+	+	Add magnitudes
+	-	Subtract magnitudes
-	+	Subtract magnitudes
-	-	Add magnitudes

Table 1: Sign Handling in Addition

Sign1	Sign2	Operation
+	+	Subtract magnitudes
+	-	Add magnitudes
-	+	Add magnitudes
-	-	Subtract magnitudes

Table 2: Sign Handling in Subtraction

Sign1	Sign2	Operation
+	+	Multiply magnitudes
+	-	Multiply magnitudes, add '-'
-	+	Multiply magnitudes, add '-'
-	-	Multiply magnitudes

Table 3: Sign Handling in Multiplication

Sign1	Sign2	Operation
+	+	Divide magnitudes
+	-	Divide magnitudes, add '-'
-	+	Divide magnitudes, add '-'
-	-	Divide magnitudes

Table 4: Sign Handling in Division

AInteger		
Visibility	Name	Type / Signature
-	number	String
+	AInteger()	Constructor
+	AInteger(String s)	Constructor
+	AInteger(AInteger original)	Constructor
+	getter()	String
+	parse(String s)	static AInteger
+	addition(AInteger, AInteger)	static AInteger
+	subtraction(AInteger, AInteger)	static AInteger
+	multiplication(AInteger, AInteger)	static AInteger
+	division(AInteger, AInteger)	static AInteger
#	cleanLeadingZeroes(String)	static String
#	reverseString(String)	static String
#	stringCompare(String, String)	static boolean
#	signDecider(String, String)	static List<String>
#	magnitudeMaker(String, String, String, String)	static List<String>
#	additionOfPositiveIntegers(String, String)	static String
#	subtractionOfPositiveIntegers(String, String)	static String
#	multiplicationOfPositiveIntegers(String, String)	static String
#	divisionOfPositiveIntegers(String, String)	static String
-	validateIntegerInput(String)	static void

Table 5: UML Table for **AInteger** Class

Attributes and Methods Defined in **AFloat**

The **AFloat** class is designed for arbitrary-precision floating-point arithmetic using string manipulation and the **AInteger** class for core operations.

Attributes

- **private String number**: Holds the floating-point value as a string for arbitrary precision. Made private to prevent accidental modification.
- **public static int precision**: Controls the number of decimal digits kept in results.

Public Methods

- **AFloat()**: Default constructor, initializes to "0".
- **AFloat(String s)**: Constructor from string.
- **AFloat(AFloat original)**: Copy constructor.
- **String getter()**: Returns the internal number string.
- **static AFloat parse(String s)**: Parses a string to an **AFloat**.
- **static void setPrecision(int precision)**: Sets the global decimal precision.
- **static AFloat addition(AFloat, AFloat)**: Adds two **AFloat** objects.
- **static AFloat subtraction(AFloat, AFloat)**: Subtracts one **AFloat** from another.
- **static AFloat multiplication(AFloat, AFloat)**: Multiplies two **AFloat** objects.
- **static AFloat division(AFloat, AFloat)**: Divides one **AFloat** by another.

Helper Functions (Mostly Private/Protected)

- `private static void validateInput(String s)`: Checks if input is a valid float string.
- `private void setter(String s)`: Sets the internal string.
- `private void truncate(int precision)`: Truncates decimals to specified precision.
- `private boolean isInt()`: Checks if the number is an integer.
- `private int min(int a, int b)`, `private static int max(int a, int b)`: functions for min/max.
- `public String integralPart()`: Returns part before decimal.
- `protected int decimalPlaces()`: Number of digits after decimal.
- `protected String decimalPart()`: Returns part after decimal.
- `protected String removeDecimal()`: Joins integral and decimal parts.
- `protected static List<String> magnitudeMaker(...)`: Returns unsigned values.
- `protected static void cleanLeadingZeroesInIntegralPart(AFloat)`: Removes leading zeros.
- `protected static String makeStandard(AFloat)`: Returns standardized forms.
- `protected static String additionOfPositiveFloat(AFloat, AFloat)`: Adds two positive floats.
- `private static String subtractionOfPositiveIntegers(String, String)`: Subtracts two positive integers as strings.
- `private static String SubtractionOfPosFloat(AFloat, AFloat)`: Subtracts two positive floats.

Why Some Methods Are Private/Protected

Helper and utility methods are private or protected to hide internal logic and prevent misuse. Only the main arithmetic operations and essential getters are public, providing a safe and simple interface.

Implementation of AFloat

Addition Simulation

1. **Align Decimal Places:** Pad the decimal part of the shorter number with zeros.

$1.2 + 3.456 \rightarrow \text{pad to } 1.200 + 3.456$

2. **Combine as Integers:**

$"1.200" \rightarrow "1200", "3.456" \rightarrow "3456"$

3. **Add Using AInteger:**

$\text{sum} = \text{AInteger.addition}("1200", "3456") \rightarrow "4656"$

4. **Reinsert Decimal:**

$"4656" \rightarrow "4.656" \text{ (3 decimal places)}$

Subtraction Simulation

1. Pad decimals to equal length.
2. Combine as integers and subtract using AInteger.
3. Reinsert decimal point.
4. Attach sign if result is negative.

Multiplication Process

"1.2" → "12", "3.4" → "34"

product = AInteger.multiplication("12", "34") → "408"

Insert decimal after (1+1)=2 digits: "4.08"

Division Algorithm

"1.2" → "12", "0.3" → "3"

Pad numerator with zeros for precision: "120000" (if precision=4)

quotient = AInteger.division("120000", "3") → "40000"

Insert decimal after 4 digits: "4.0000"

Sign Handling Tables

Sign1	Sign2	Operation
+	+	Add magnitudes
+	-	Subtract magnitudes
-	+	Subtract magnitudes
-	-	Add magnitudes

Table 6: Sign Handling in Addition

Sign1	Sign2	Operation
+	+	Subtract magnitudes
+	-	Add magnitudes
-	+	Add magnitudes
-	-	Subtract magnitudes

Table 7: Sign Handling in Subtraction

Sign1	Sign2	Operation
+	+	Multiply magnitudes
+	-	Multiply magnitudes, add '-'
-	+	Multiply magnitudes, add '-'
-	-	Multiply magnitudes

Table 8: Sign Handling in Multiplication

Sign1	Sign2	Operation
+	+	Divide magnitudes
+	-	Divide magnitudes, add '-'
-	+	Divide magnitudes, add '-'
-	-	Divide magnitudes

Table 9: Sign Handling in Division

AFloat		
Visibility	Name	Type / Signature
-	number	String
+	precision	static int
+	AFloat()	Constructor
+	AFloat(String s)	Constructor
+	AFloat(AFloat original)	Constructor
+	getter()	String
+	parse(String s)	static AFloat
+	setPrecision(int)	static void
+	addition(AFloat, AFloat)	static AFloat
+	subtraction(AFloat, AFloat)	static AFloat
+	multiplication(AFloat, AFloat)	static AFloat
+	division(AFloat, AFloat)	static AFloat
-	validateInput(String)	static void
-	setter(String)	void
-	truncate(int)	void
-	isInt()	boolean
-	min(int, int)	int
-	max(int, int)	static int
+	integralPart()	String
#	decimalPlaces()	int
#	decimalPart()	String
#	removeDecimal()	String
#	magnitudeMaker(String, String, String, String)	static List<String>
#	cleanLeadingZeroesInIntegralPart(AFloat)	static void
#	makeStandard(AFloat)	static String
#	additionOfPositiveFloat(AFloat, AFloat)	static String
-	subtractionOfPositiveIntegers(String, String)	static String
-	SubtractionOfPosFloat(AFloat, AFloat)	static String

Table 10: UML Table for AFloat Class

How to Compile and Run MyInfArith.java Using Ant

To build and run your MyInfArith.java program using Ant, follow these steps in your terminal:

1. **Clean the project (remove old class files and jar):**

```
ant clean
```

2. **Compile all Java source files:**

```
ant compile
```

3. **Create the jar file for the arithmetic library:**

```
ant jar
```

4. **Run the main program (MyInfArith.java):**

```
ant run
```

You can give input in the format `ant run -Dargs="float div 5.5 2"`

Note: Make sure your `build.xml` file is in the root directory of your project, and that you have Apache Ant installed on your system.

You can also run your program using the Python script with a single command. For example:

How to Run MyInfArith.java Using python script "CalcEngine"

you can run by doing the following

```
python CalcEngine.py <int/float> <add/sub/mul/div> op1 op2
```

Note: Make sure the class files of AFloat.java , MyInfArith.java and AInteger.java are already present or else it won't work.

Verification Approach

To ensure whether my package was working perfectly fine , I used some large numerical computations for which the correct answers were already known to me. The CalcEngine calculated the outputs of all correctly when precision is set to 30(default).

- **Input:** java MyInfArith int add 23650078224912949497310933240250 42939783262467113798386384401498
Output: 66589861487380063295697317641748
- **Input:** java MyInfArith int sub 3116511674006599806495512758577 57745242300346381144446453884008
Output: -54628730626339781337950941125431
- **Input:** java MyInfArith int mul 14344163160445929942680697312322 23017167694823904478474013730519
Output: 330162008905899217578310782382075660760972861550182008086155118
- **Input:** java MyInfArith int div 8792726365283060579833950521677211 493835253617089647454998358
Output: 17804979
- **Input:** java MyInfArith float div 8792726365283060579833950521677211.0 493835253617089647454998358
Output: 17804979.091469989302961159520087878533
- **Input:** java MyInfArith float add 84486723.420039 70974199.843732
Output: 155460923.263771
- **Input:** java MyInfArith float sub 840196454.51725 712586963.70283
Output: 127609490.81442
- **Input:** java MyInfArith float mul 6400251.9377695 2326541.6827934
Output: 14890452913599.9717457253213
- **Input:** java MyInfArith float div 244727.15202 75964.3891
Output: 3.221603634537752111008551505615
- **Input:** java MyInfArith int div 25 123
Output: 0
- **Input:** java MyInfArith float div 3227 555
Output: 5.8144144... (repeating up to 30 digits)
- **Input:** java MyInfArith float div 5.5 2
Output: 2.75
- **Input:** java MyInfArith int div 2 0
Output: Division by zero error

All these test cases produced the correct results, even for very large numbers. The package also correctly handles edge cases, such as division by zero, by returning an appropriate error message.

Key Learnings

- Learnt how to use jar and Ant.
- Gained more insights about Object-Oriented Programming (OOP) in Java.
- Git usage.
- Docker usage.

Limitations

- Cannot parse in scientific notations
- Not optimized for best possible memory and time efficiency
- Lacks built-in functions like square, cube, etc.
- This calculator doesn't round off instead it will truncate to the precision.